RW 연산 (Read-Write Operations)

개요

RW 연산은 EVM 실행 중 발생하는 모든 상태 읽기/쓰기 작업을 추적하는 핵심 메커니즘입니다. Bus-Mapping 단계에서 생성된 RW 연산들은 State Circuit에서 검증되며, 각 연산은 고유한 RW Counter로 식별됩니다.

주요 역할:

- EVM 실행 중 모든 상태 변경 추적
- 스택, 메모리, 스토리지 등의 읽기/쓰기 기록
- State Circuit의 입력 데이터 생성
- 연산 순서 보장 및 일관성 검증

파일 위치:

- `bus-mapping/src/operation.rs` - RW 연산 타입 정의
- `bus-mapping/src/operation/container.rs` - 연산 컨테이너
- `bus-mapping/src/circuit_input_builder/input_state_ref.rs` - 연산 생성 로직

RW 연산 기본 구조

RW 마커

모든 연산은 읽기 또는 쓰기로 분류됩니다:

```
pub enum RW {
```

```
READ, // 읽기 연산
WRITE, // 쓰기 연산
}
```

메서드:

- `is_read()` : 읽기 연산인지 확인
- `is_write()` : 쓰기 연산인지 확인

RWCounter (RW 카운터)

RWCounter는 각 연산에 고유한 순서를 부여하는 전역 카운터입니다:

```
pub struct RWCounter(pub usize);
```

특징:

- 초기값: 1 (0이 아님)
- 각 연산마다 1씩 증가
- 연산의 실행 순서를 보장
- State Circuit에서 정렬 키로 사용

주요 메서드:

```
impl RWCounter {
pub fn new() -> Self { Self(1) }
pub fn inc(&mut self) { self.0 += 1; }
pub fn inc_pre(&mut self) -> Self {
let pre = *self;
```

```
self.inc();

pre

}

}
```

Operation 래퍼

모든 RW 연산은 `Operation<T>` 래퍼로 감싸집니다:

```
pub struct Operation<T: Op> {

rwc: RWCounter, // RW 카운터

rw: RW, // 읽기/쓰기 구분

reversible: bool, // revert 시 되돌릴 수 있는지 여부

op: T, // 실제 연산 데이터

}
```

생성 메서드:

- `Operation::new(rwc, rw, op)` : 일반 연산 생성
- `Operation::new_reversible(rwc, rw, op)` : 되돌릴 수 있는 연산 생성

RW 연산 타입 (Target)

RW 연산은 다음 타입들로 분류됩니다:

```
pub enum Target {
```

```
Start, // 패딩용 시작 연산
Memory, // 메모리 연산
Stack, // 스택 연산
Storage, // 영구 스토리지 연산
TransientStorage, // 임시 스토리지 연산 (EIP-1153)
TxAccessListAccount, // 트랜잭션 액세스 리스트 (계정)
TxAccessListAccountStorage, // 트랜잭션 액세스 리스트 (스토리지)
TxRefund, // 트랜잭션 환불
Account, // 계정 상태
CallContext, // 호출 컨텍스트
TxReceipt, // 트랜잭션 영수증
TxLog, // 트랜잭션 로그
}
```

개별 RW 연산 타입

1. StackOp (스택 연산)

스택 읽기/쓰기를 추적합니다:

```
pub struct StackOp {
pub call_id: usize, // 호출 ID
pub address: StackAddress, // 스택 주소 (1024부터 시작)
pub value: Word, // 256비트 값
```

```
}
```

특징:

- 스택은 1024부터 시작하여 아래로 증가 (1024, 1023, 1022, ...)
- 각 호출마다 독립적인 스택 공간
- 읽기와 쓰기 모두 발생
- 되돌릴 수 없음 (non-reversible)

정렬 기준: `(call_id, address)`

2. MemoryOp (메모리 연산)

메모리 읽기/쓰기를 추적합니다:

```
pub struct MemoryOp {
pub call_id: usize, // 호출 ID
pub address: MemoryAddress, // 메모리 주소 (바이트 단위)
pub value: Word, // 현재 값 (256비트)
pub value_prev: Word, // 이전 값
}
```

특징:

- 메모리는 바이트 단위로 주소 지정
- Word 단위(32바이트)로 읽기/쓰기
- 각 호출마다 독립적인 메모리 공간
- 초기값은 0

- 되돌릴 수 없음 (non-reversible)

정렬 기준: `(call_id, address)`

3. StorageOp (스토리지 연산)

영구 스토리지 읽기/쓰기를 추적합니다:

```
pub struct StorageOp {
pub address: Address, // 계정 주소
pub key: Word, // 스토리지 키 (256비트)
pub value: Word, // 현재 값
pub value_prev: Word, // 이전 값
pub tx_id: usize, // 트랜잭션 ID
pub committed_value: Word, // 트랜잭션 시작 시 값
}
```

특징:

- 계정별 영구 저장소
- SLOAD/SSTORE 명령어로 접근
- 가스 비용이 `committed_value` 에 따라 달라짐
- 되돌릴 수 있음 (reversible)
- `reverse()` 메서드로 `value` 와 `value_prev` 교환

정렬 기준: `(address, key)`

4. TransientStorageOp (임시 스토리지 연산)

EIP-1153에서 도입된 임시 스토리지 연산:

```
pub struct TransientStorageOp {
pub address: Address, // 계정 주소
pub key: Word, // 스토리지 키
pub value: Word, // 현재 값
pub value_prev: Word, // 이전 값
pub tx_id: usize, // 트랜잭션 ID
}
```

특징:

- 트랜잭션 내에서만 유효 (트랜잭션 종료 시 삭제)
- TLOAD/TSTORE 명령어로 접근
- 가스 비용이 영구 스토리지보다 저렴
- 되돌릴 수 있음 (reversible)

정렬 기준: `(address, key)`

5. TxAccessListAccountOp (계정 액세스 리스트)

계정의 WARM/COLD 상태를 추적합니다:

```
pub struct TxAccessListAccountOp {
pub tx_id: usize, // 트랜잭션 ID
pub address: Address, // 계정 주소
pub is_warm: bool, // 현재 WARM 상태
pub is_warm_prev: bool, // 이전 WARM 상태
```

```
}
```

특징:

- EIP-2929에서 도입
- 첫 접근은 COLD (비용 높음), 이후는 WARM (비용 낮음)
- BALANCE, EXTCODESIZE, EXTCODEHASH, CALL 등에서 사용
- 되돌릴 수 있음 (reversible)

정렬 기준: `(tx_id, address)`

6. TxAccessListAccountStorageOp (스토리지 액세스 리스트)

스토리지 슬롯의 WARM/COLD 상태를 추적합니다:

```
pub struct TxAccessListAccountStorageOp {
pub tx_id: usize, // 트랜잭션 ID
pub address: Address, // 계정 주소
pub key: Word, // 스토리지 키
pub is_warm: bool, // 현재 WARM 상태
pub is_warm_prev: bool, // 이전 WARM 상태
}
```

특징:

- SLOAD/SSTORE의 가스 비용 결정
- 첫 접근은 COLD, 이후는 WARM
- 되돌릴 수 있음 (reversible)

정렬 기준: `(tx_id, address, key)`

7. TxRefundOp (트랜잭션 환불)

가스 환불을 추적합니다:

```
pub struct TxRefundOp {
pub tx_id: usize, // 트랜잭션 ID
pub value: u64, // 현재 환불량
pub value_prev: u64, // 이전 환불량
}
```

특징:

- SSTORE에서 스토리지를 0으로 설정할 때 환불
- SELFDESTRUCT에서 환불
- 되돌릴 수 있음 (reversible)

정렬 기준: `(tx_id)`

8. AccountOp (계정 연산)

계정 필드의 변경을 추적합니다:

```
pub struct AccountOp {
pub address: Address, // 계정 주소
pub field: AccountField, // 필드 타입
pub value: Word, // 현재 값
pub value_prev: Word, // 이전 값
```

```
}

pub enum AccountField {
Nonce, // 계정 논스
Balance, // 계정 잔액
CodeHash, // Poseidon 코드 해시
KeccakCodeHash, // Keccak 코드 해시
CodeSize, // 코드 크기
}
```

특징:

- 계정 상태 변경 추적
- CREATE, CALL, SELFDESTRUCT 등에서 발생
- 되돌릴 수 있음 (reversible)

정렬 기준: `(address, field)`

9. CallContextOp (호출 컨텍스트)

호출 컨텍스트 정보를 추적합니다:

```
pub struct CallContextOp {
pub call_id: usize, // 호출 ID
pub field: CallContextField, // 필드 타입
pub value: Word, // 값
}
```

```rust
pub  enum CallContextField {
RwCounterEndOfReversion, // Revert 종료 시 RW 카운터
CallerId, // 호출자 ID
TxId, // 트랜잭션 ID
Depth, // 호출 깊이
CallerAddress, // 호출자 주소
CalleeAddress, // 피호출자 주소
CallDataOffset, // CallData 오프셋
CallDataLength, // CallData 길이
ReturnDataOffset, // ReturnData 오프셋
ReturnDataLength, // ReturnData 길이
Value, // 전송 값
IsSuccess, // 성공 여부
IsPersistent, // 영구 여부
IsStatic, // Static 호출 여부
LastCalleeId, // 마지막 피호출자 ID
LastCalleeReturnDataOffset, // 마지막 ReturnData 오프셋
LastCalleeReturnDataLength, // 마지막 ReturnData 길이
IsRoot, // 루트 호출 여부
IsCreate, // CREATE 호출 여부
CodeHash, // 코드 해시
ProgramCounter, // 프로그램 카운터
```

```
StackPointer, // 스택 포인터
GasLeft, // 남은 가스
MemorySize, // 메모리 크기
ReversibleWriteCounter, // 되돌릴 수 있는 쓰기 카운터
L1Fee, // L1 수수료
}
```

특징:

- 호출 실행 중 필요한 컨텍스트 정보
- 읽기 전용 (되돌릴 수 없음)
- EVM Circuit에서 광범위하게 사용

정렬 기준: `(call_id, field)`

10. TxReceiptOp (트랜잭션 영수증)

트랜잭션 영수증 정보를 추적합니다:

```
pub struct TxReceiptOp {
pub tx_id: usize, // 트랜잭션 ID
pub field: TxReceiptField, // 필드 타입
pub value: u64, // 값
}

pub enum TxReceiptField {
PostStateOrStatus, // 성공/실패 상태
```

```
CumulativeGasUsed, // 누적 가스 사용량
LogLength, // 로그 개수
}
```

특징:

- 트랜잭션 종료 시 생성
- 읽기 전용 (되돌릴 수 없음)

정렬 기준: `(tx_id, field)`

11. TxLogOp (트랜잭션 로그)

LOG0~LOG4 명령어로 생성된 로그를 추적합니다:

```
pub struct TxLogOp {
pub tx_id: usize, // 트랜잭션 ID
pub log_id: usize, // 로그 ID (1부터 시작)
pub field: TxLogField, // 필드 타입
pub index: usize, // 토픽/데이터 인덱스
pub value: Word, // 값
}

pub enum TxLogField {
Address, // 컨트랙트 주소
Topic, // 로그 토픽
Data, // 로그 데이터
```

```
}
```

특징:

- LOG0, LOG1, LOG2, LOG3, LOG4 명령어에서 생성
- 토픽은 최대 4개
- 읽기 전용 (되돌릴 수 없음)

정렬 기준: `(tx_id, log_id, field)`

12. StartOp (시작 연산)

패딩용 시작 연산:

```
pub struct StartOp {}
```

특징:

- RW 테이블의 패딩용
- 실제 연산이 아님

OperationContainer (연산 컨테이너)

모든 RW 연산은 `OperationContainer` 에 저장됩니다:

```
pub struct OperationContainer {
pub memory: Vec<Operation<MemoryOp>>,
pub stack: Vec<Operation<StackOp>>,
pub storage: Vec<Operation<StorageOp>>,
```

```rust
pub transient_storage: Vec<Operation<TransientStorageOp>>,
pub tx_access_list_account: Vec<Operation<TxAccessListAccountOp>>,
pub tx_access_list_account_storage: Vec<Operation<TxAccessListAccountStorageOp>
pub tx_refund: Vec<Operation<TxRefundOp>>,
pub account: Vec<Operation<AccountOp>>,
pub call_context: Vec<Operation<CallContextOp>>,
pub tx_receipt: Vec<Operation<TxReceiptOp>>,
pub tx_log: Vec<Operation<TxLogOp>>,
pub start: Vec<Operation<StartOp>>,
}
```

주요 메서드:

insert()

연산을 컨테이너에 삽입하고 참조를 반환합니다:

```rust
pub fn insert<T: Op>(&mut self, op: Operation<T>) -> OperationRef
```

동작:

1. 연산 타입에 따라 적절한 벡터에 추가
2. `reversible` 플래그에 따라 `Operation::new()` 또는 `Operation::new_reversible()` 사용
3. `OperationRef` 반환 (타입과 인덱스 포함)

정렬 메서드

각 연산 타입별로 정렬된 벡터를 반환합니다:

```
pub fn sorted_stack(&self) -> Vec<Operation<StackOp>>
pub fn sorted_memory_word(&self) -> Vec<Operation<MemoryOp>>
pub fn sorted_storage(&self) -> Vec<Operation<StorageOp>>
```

정렬 기준:

- 각 연산 타입의 `Ord` 구현에 따름
- 동일한 키의 경우 `rw_counter` 순서

RW 연산 생성 프로세스

1. push_op() - 일반 연산 추가

`CircuitInputStateRef` 의 `push_op()` 메서드로 연산을 추가합니다:

```
pub fn push_op<T: Op>(
&mut self,
step: &mut ExecStep,
rw: RW,
op: T
) -> Result<(), Error>
```

동작 순서:

1. 현재 `RWCounter` 값 가져오기
2. `Operation::new(rwc, rw, op)` 생성

3. `OperationContainer::insert()` 호출
4. 반환된 `OperationRef` 를 `ExecStep` 의 `bus_mapping_instance` 에 추가
5. `RWCounter` 증가 (`inc_pre()` 사용)
6. RW 개수 제한 확인

예시 (SLOAD):

```
// 스토리지 읽기
state.push_op(
&mut exec_step,
RW::READ,
StorageOp::new(
contract_addr,
key,
value,
value,
tx_id,
committed_value,
),
)?;
```

2. push_op_reversible() - 되돌릴 수 있는 연산 추가

되돌릴 수 있는 쓰기 연산을 추가합니다:

```
pub fn push_op_reversible<T: Op>(
```

```
&mut self,
step: &mut ExecStep,
op: T
) -> Result<(), Error>
```

동작 순서:

1. 현재 `RWCounter` 값 가져오기
2. `Operation::new_reversible(rwc, RW::WRITE, op)` 생성
3. `OperationContainer::insert()` 호출
4. 반환된 `OperationRef` 를 `ExecStep` 의 `bus_mapping_instance` 에 추가
5. `RWCounter` 증가
6. `reversible_write_counter` 증가
7. 호출이 persistent하지 않으면 `reversion_groups` 에 추가
8. RW 개수 제한 확인

예시 (SSTORE):

```
// 스토리지 쓰기 (되돌릴 수 있음)
state.push_op_reversible(
&mut exec_step,
StorageOp::new(
contract_addr,
key,
value,
```

```
value_prev,
tx_id,
committed_value,
),
)?;
```

3. 특수 메서드

call_context_read()

CallContext 읽기 전용 메서드:

```
pub fn call_context_read(
&mut self,
step: &mut ExecStep,
call_id: usize,
field: CallContextField,
value: Word,
) -> Result<(), Error>
```

call_context_write()

CallContext 쓰기 메서드:

```
pub fn call_context_write(
&mut self,
step: &mut ExecStep,
```

```
call_id: usize,
field: CallContextField,
value: Word,
) -> Result<(), Error>
```

RWCounter 관리

초기화

블록 처리 시작 시 `RWCounter` 는 1로 초기화됩니다:

```
let mut rwc = RWCounter::new(); // RWCounter(1)
```

증가 패턴

`inc_pre()` 메서드를 사용하여 현재 값을 반환하고 증가합니다:

```
let current_rwc = rwc.inc_pre(); // 현재 값 반환 후 증가
// rwc는 이제 다음 값을 가리킴
```

이 패턴은 다음을 보장합니다:

1. 각 연산이 고유한 RWCounter를 가짐
2. 연산 순서가 RWCounter 순서와 일치
3. 동시에 여러 연산을 생성해도 순서 보장

제한 확인

RW 연산 개수는 회로 파라미터로 제한됩니다:

```
pub fn check_rw_num_limit(&self) -> Result<(), Error> {
let max_rws = self.block.circuits_params.max_rws;
let effective_limit = if max_rws == 0 {
1_000_000 // 동적 모드에서도 최대 1M
} else {
max_rws
};
let rwc = self.block_ctx.rwc.0;

if rwc > effective_limit && cfg!(feature = "strict-ccc") {
return Err(Error::InternalError("rws not enough"));
}
Ok(())
}
```

연산 순서 보장 메커니즘

1. RWCounter 순서

모든 연산은 생성 순서대로 증가하는 RWCounter를 가집니다:

```
연산 1: RWCounter(1)
연산 2: RWCounter(2)
```

```
연산 3: RWCounter(3)

...
```

2. ExecStep 내 순서

각 `ExecStep` 은 `bus_mapping_instance` 에 `OperationRef` 리스트를 유지합니다:

```rust
pub struct ExecStep {
pub bus_mapping_instance: Vec<OperationRef>,
// ...
}
```

이를 통해:

- 각 실행 단계에서 발생한 연산 추적
- 연산 순서 재구성 가능
- 디버깅 및 검증 용이

3. State Circuit 정렬

State Circuit에서는 다음 순서로 정렬됩니다:

1. **1차 정렬:** 연산 타입별 정렬 키 (tag, id, address, field_tag, storage_key)
2. **2차 정렬:** 동일한 키 내에서 RWCounter 순서

예시:

```
Stack(call_id=1, addr=1023, rwc=5)
Stack(call_id=1, addr=1023, rwc=8)
```

```
Stack(call_id=1, addr=1022, rwc=6)

Memory(call_id=1, addr=0, rwc=7)

Storage(addr=0x123..., key=0x1, rwc=9)
```

정렬 후:

```
Memory(call_id=1, addr=0, rwc=7)

Stack(call_id=1, addr=1022, rwc=6)

Stack(call_id=1, addr=1023, rwc=5)

Stack(call_id=1, addr=1023, rwc=8)

Storage(addr=0x123..., key=0x1, rwc=9)
```

Reversible 연산과 Reversion 처리

Reversible 연산

다음 연산들은 revert 시 되돌릴 수 있습니다:

- `StorageOp`
- `TransientStorageOp`
- `TxAccessListAccountOp`
- `TxAccessListAccountStorageOp`
- `TxRefundOp`
- `AccountOp`

Reversion Groups

Non-persistent 호출에서 발생한 reversible 연산들은 `reversion_groups` 에 저장됩니다:

```
pub struct ReversionGroup {
pub op_refs: Vec<(usize, OperationRef)>, // (step_index, op_ref)
}
```

Revert 처리

호출이 revert될 때:

1. 해당 `reversion_group` 의 모든 연산 찾기
2. 각 연산의 `reverse()` 메서드 호출
3. 역방향 연산을 새로운 RWCounter로 추가

예시:

```
// 원래 연산
StorageOp {
value: 100,
value_prev: 50,
rwc: 10,
}

// Revert 시 생성되는 역방향 연산
StorageOp {
value: 50, // value_prev와 교환
value_prev: 100, // value와 교환
```

```
rwc: 15, // 새로운 RWCounter
}
```

RwCounterEndOfReversion

각 호출은 `RwCounterEndOfReversion` 필드를 가집니다:

```
CallContextOp {
call_id: 1,
field: CallContextField::RwCounterEndOfReversion,
value: 20, // Revert 종료 시점의 RWCounter
}
```

이 값은:

- 호출 시작 시 placeholder로 설정
- 블록 처리 완료 후 실제 값으로 업데이트
- State Circuit에서 reversion 범위 검증에 사용

연산 생성 예시

SSTORE 명령어

SSTORE는 여러 RW 연산을 생성합니다:

```
// 1. 스토리지 쓰기 (reversible)
state.push_op_reversible(
```

```
&mut  exec_step,
StorageOp::new(
contract_addr,
key,
value,
value_prev,
tx_id,
committed_value,
),
)?;


// 2. 액세스 리스트 읽기
state.push_op(
&mut  exec_step,
RW::READ,
TxAccessListAccountStorageOp {
tx_id,
address: contract_addr,
key,
is_warm: true,
is_warm_prev: false,
},
)?;
```

```
// 3. 액세스 리스트 쓰기 (reversible)
state.push_op_reversible(
&mut  exec_step,
TxAccessListAccountStorageOp {
tx_id,
address: contract_addr,
key,
is_warm: true,
is_warm_prev: false,
},
)?;


// 4. 환불 업데이트 (reversible, 조건부)
if  should_refund {
state.push_op_reversible(
&mut  exec_step,
TxRefundOp {
tx_id,
value: new_refund,
value_prev: old_refund,
},
)?;
```

```
}
```

CALL 명령어

CALL은 CallContext 연산을 생성합니다:

```
// 호출자 컨텍스트 읽기
state.call_context_read(
&mut  exec_step,
caller_call_id,
CallContextField::TxId,
tx_id.into(),
)?;

// 새 호출 컨텍스트 쓰기
state.call_context_write(
&mut  exec_step,
callee_call_id,
CallContextField::CallerId,
caller_call_id.into(),
)?;

state.call_context_write(
&mut  exec_step,
```

```
callee_call_id,
CallContextField::IsSuccess,
Word::from(is_success  as u64),
)?;


// 계정 액세스 리스트 업데이트 (reversible)
state.push_op_reversible(
&mut  exec_step,
TxAccessListAccountOp {
tx_id,
address: callee_address,
is_warm: true,
is_warm_prev: false,
},
)?;
```

LOG 명령어

LOG는 TxLog 연산을 생성합니다:

```
// 로그 주소
state.push_op(
&mut  exec_step,
RW::WRITE,
```

```
TxLogOp::new(
tx_id,
log_id,
TxLogField::Address,
0,
contract_addr.to_word(),
),
)?;


// 토픽들
for (topic_index, topic) in  topics.iter().enumerate() {
state.push_op(
&mut  exec_step,
RW::WRITE,
TxLogOp::new(
tx_id,
log_id,
TxLogField::Topic,
topic_index,
*topic,
),
)?;
}
```

```
// 데이터 바이트들
for (byte_index, byte) in  data.iter().enumerate() {
state.push_op(
&mut  exec_step,
RW::WRITE,
TxLogOp::new(
tx_id,
log_id,
TxLogField::Data,
byte_index,
Word::from(*byte),
),
)?;
}
```

State Circuit과의 연동

RW Table 생성

`OperationContainer` 의 모든 연산은 State Circuit의 RW Table로 변환됩니다:

```
// 각 연산 타입별로 정렬
let  sorted_stack = container.sorted_stack();
```

```
let sorted_memory = container.sorted_memory_word();
let sorted_storage = container.sorted_storage();
// ...


// RW Table 행 생성
for op in all_operations {
rw_table.push(RwRow {
rw_counter: op.rwc(),
is_write: op.rw().is_write(),
tag: op.target(),
// ... 연산별 필드 매핑
});
}
```

정렬 및 검증

State Circuit은 다음을 검증합니다:

1. **Lexicographic Ordering**: 모든 연산이 올바른 순서로 정렬됨
2. **Read-Write Consistency**: 읽기 값이 이전 쓰기 값과 일치
3. **Initial Values**: 첫 읽기의 초기값이 올바름
4. **MPT Updates**: 스토리지 변경이 MPT와 일치

최적화 및 고려사항

1. RW 개수 최소화

불필요한 RW 연산을 줄이기 위한 전략:

- **중복 제거**: 동일한 값을 여러 번 읽지 않음
- **배치 처리**: 여러 연산을 한 번에 처리
- **캐싱**: 자주 사용되는 값 캐싱

2. 메모리 효율

`OperationContainer` 는 각 타입별로 별도 벡터를 유지하여:

- 타입별 정렬 효율성 향상
- 메모리 레이아웃 최적화
- 캐시 친화적 접근

3. 정렬 성능

정렬은 비용이 큰 작업이므로:

- 삽입 시 정렬하지 않고 벡터에 추가
- 필요할 때만 `sorted_*()` 메서드로 정렬
- 정렬 결과 캐싱 가능

4. Reversible 연산 추적

Reversible 연산은 추가 메타데이터를 유지:

- `reversible_write_counter` : 되돌릴 수 있는 쓰기 개수
- `reversion_groups` : Revert 시 되돌릴 연산 목록
- 메모리 오버헤드와 정확성 사이의 균형

디버깅 팁

1. RW 연산 추적

각 ExecStep의 `bus_mapping_instance` 를 확인하여 어떤 연산이 생성되었는지 추적:

```
for (i, op_ref) in  exec_step.bus_mapping_instance.iter().enumerate() {
println!("Op {}: {:?}", i, op_ref);
}
```

2. RWCounter 불일치

RWCounter가 예상과 다르면:

- `inc_pre()` 호출 순서 확인
- 조건부 연산 생성 로직 확인
- Revert 처리 확인

3. 정렬 오류

State Circuit에서 정렬 오류가 발생하면:

- 각 연산의 정렬 키 확인
- `Ord` 구현 검증
- 동일 키의 RWCounter 순서 확인

4. Reversible 연산 누락

Revert 시 상태가 올바르게 복원되지 않으면:

- `push_op_reversible()` 사용 여부 확인

- `reversion_groups` 에 올바르게 추가되었는지 확인
- `reverse()` 메서드 구현 확인

관련 문서

- Bus-Mapping 개요 - Bus-Mapping 시스템 전체 구조
- 트레이스 수집 - Geth 트레이스 수집 과정
- Opcode 처리 - Opcode별 RW 연산 생성
- State Circuit - RW 테이블 검증
- RW Table - RW 테이블 스키마

참고 자료

코드 파일:

- `bus-mapping/src/operation.rs` - RW 연산 타입 정의
- `bus-mapping/src/operation/container.rs` - 연산 컨테이너
- `bus-mapping/src/circuit_input_builder/input_state_ref.rs` - 연산 생성 로직
- `bus-mapping/src/evm/opcodes/` - Opcode별 연산 생성

관련 EIP:

- EIP-2929: Gas cost increases for state access opcodes
- EIP-1153: Transient storage opcodes
- EIP-2930: Optional access lists